Data Scientist - Technical Test: Transaction Labeling

Introduction

This technical test evaluates how you approach a common Personal Finance Management (PFM) problem: **classifying card transactions into standardized categories** (e.g. supermarkets, gas stations, betting…) from the information available at transaction time.

You will work with a labeled dataset of historical transactions and build a model that, given a **single new transaction as input, returns its predicted category**, exposed via a small prediction API.

Please make your choices explicit (assumptions, trade-offs, error analysis) and keep the solution reproducible. Feel free to make reasonable assumptions where needed.

We advise you not to spend more than half a day on this test. We understand that this is already a long assignment. **Do not worry when submitting your work if it is not perfect or finished, but do mention how you imagined the rest.**

Dataset

We provide a single **annotated synthetic CSV file** of card transactions. Each row is one transaction and includes the information typically available at transaction time (transaction timestamp, merchant information, amounts, currency, etc.).

CSV data dictionary

Column name	Type	Description
transaction_id	string	Unique identifier of the transaction (primary key)
ref_id	string	Internal technical reference of the transaction
created_at	datetime (ISO 8601)	Transaction creation timestamp
member_id	int	Customer identifier
member_name	string	Customer display name
merchant_id	string	Merchant / point-of-sale identifier
merchant_name	string	Merchant name as it appears on the transaction
title	string	Transaction label as displayed to the customer
amount_value	number	Signed amount
amount_currency	string	ISO currency code
fees	int	Fees applied, in cents
card_type	string	Card product type
card_name	string	Card commercial name

Column name	Type	Description
digitalization_type	string	Tokenization / wallet used
merchant_pan_entry_method	string	How the card number was captured
pfm_recurring_frequency	string	Detected recurrence
status	string	Transaction status
major_category	string	**Target** : coarse category
minor_category	string	**Target** : fine-grained category

Categories

Each transaction has two label levels to predict:

`minor_category` fine-grained category (12 classes):

- `supermarkets`
- `multimedia`
- `restaurants`
- `betting`
- `tolls_gas_stations`
- `atm`
- `refund`
- `hairdresser`
- `others_shopping`
- `pharmacies`
- `transport_means`
- `travel_means`

`major_category` coarse category (8 classes):

- `groceries`
- `entertainment`
- `restaurants_bars_cafes`
- `transportation`
- `finance`
- `self_care`
- `shopping`
- `travel`

Requirements

Please share your work in up to 7 days to `gabriel.baker@lydia-app.com` and `camille.omnes@lydia-app.com` as a link to a **Git repository.**

In that email please provide the answers to:

- How much time did you allocate for this test?
  - ☐ Less than 2 hours
  - ☐ Between 2 and 4 hours
  - ☐ More than 4 hours: ____ hours
- How would you rate the overall quality of your submission?
  - ☐ Poor
  - ☐ OK
  - ☐ Good
  - ☐ Excellent

Deliverables

1. **Your source code**, including:
   - Any exploratory analysis performed on the data
   - Model training
   - The prediction service (see point 2)
2. **A prediction API** runnable locally that exposes a way to classify a single new transaction (one transaction in → predicted category out). You are free to choose the framework, the language, and the exact contract; we just need to be able to run it on our side and feed it a transaction.
3. **A short write-up (README)** describing:
   - Your approach and key design choices
   - Result analysis
   - How to run training
   - How to run the prediction service (install, start command, example of a request and the format expected)

Evaluation Criteria

We will primarily evaluate **your approach and execution quality** across:

- **Methodology**: framing, assumptions, baseline + iteration.
- **Exploration**: analysis done prior to training.

- **Engineering**: code quality, reproducibility, clear run instructions.
- **API**: understand of concepts of deploying models in production.
- **Communication**: concise README with decisions and trade-offs.